人工知能学会第二種研究会資料
汎用人工知能研究会
SIG-AGI-027-02

プログラム合成対象言語 Pro5Lang における知識表現形式

Knowledge Representation Format for the Program Synthesis Target Language Pro5Lang

一杉裕志 [1*]　高橋直人 [1]　竹内泉 [1]　佐野崇 [2]　中田秀基 [3]
Yuuji Ichisugi[1]　Naoto Takahashi[1]　Izumi Takeuti[1]　Takashi Sano[2]　Hidemoto Nakada[3]

[1] 産業技術総合研究所 [1] National Institute of Advanced Industrial Science and Technology
[2] 東洋大学　[2] Toyo University　[3] 順天堂大学　[3] Juntendo University

Abstract: Program synthesis is one promising approach towards realizing AGI. The performance of program synthesis depends on the language specifications of the target language. We have mainly designed the control structure of the target language Pro5Lang, but have not given sufficient consideration to the method of expressing data structures. In this paper, inspired by the human brain, we attempt to design a knowledge representation format that combines high expressiveness with ease of program synthesis. We wrote a test program using the proposed knowledge representation format to perform reasoning using proof by contradiction and proof by cases, and confirmed the expressiveness of the language.

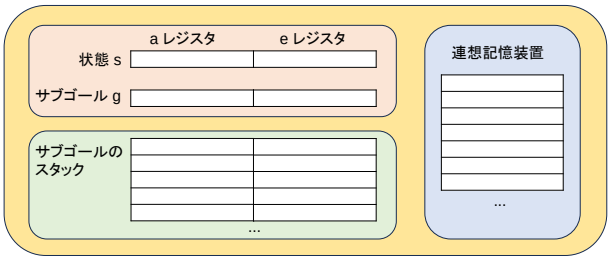

図 1: Pro5Lang のメモリ構成。状態 s とサブゴール g の組は機械語のプログラムカウンタに相当し、次に実行する行動ルールの選択に使われる。レジスタと連想記憶装置に入れられる値はすべて図 2 の形式で表現された証明済み命題である。

1　はじめに

我々はこれまで、再帰的強化学習・生成モデル・プログラム合成の３つを中核技術とした脳型 AGI アーキテクチャの早期実現を目指す研究 [1] を行ってきた。設計中の脳型 AGI アーキテクチャは、Pro5Lang [2] と呼ぶプログラミング言語のプログラムを、強化学習を用いて自律的に合成する。

我々はこれまで Pro5Lang の実行制御に関する機構（条件分岐、サブルーチン、例外処理 [3][4]）と連想記憶装置の機構 [2] の設計を行ってきた。データの表現方法に関する考察 [5] も行ったが、これまでの Pro5Lang は複雑なデータ構造を表現する機構を持っておらず、社会のルール、他者の心理、数学や自然科学の法則など、人間が扱う多様な知識を表現し処理する能力はなかった。言うまでもなく、これらは AGI に求められる重要な能力である。

本研究では、ヒトの脳にヒントを得て、高い表現力とプログラム合成のしやすさを兼ね備えた知識表現形式の設計を試みた。提案する知識表現形式は、様々な研究分野におけるヒトの知識表現や推論に関する過去の議論を参考にして設計されている。大きく関連するのは、記号 AI における状況意味論 [6][7] と、数理論理学における自然演繹 [8][9] である[1]。これら関連技術との関係については 6 節で詳しく議論する。

提案する知識表現形式を用いて背理法と場合分けを用いた推論を行うテストプログラムを記述し、言語の表現力について確認した。これらは現実の命題と仮想的状況における命題が混在した推論であり、ヒトも自然に行える推論である。

本稿は以下のような構成になっている。2 節で合成対象言語 Pro5Lang の説明、3 節で提案する知識表現形式の説明、4 節でテスト用タスクの説明を行った後、5 節で議論を行い、6 節で関連研究について述べ、最後に 7 節でまとめを述べる。

*連絡先：産業技術総合研究所
茨城県つくば市東１－１－１ 中央事業所
E-mail: y-ichisugi@aist.go.jp

[1] 他には、記号 AI における状況計算、 RDF (Resource Description Framework)、認知科学における状況モデル、数理論理学における可能世界意味論、 Lisp, Prolog, Java などのプログラミング言語などを参考にしている。

2 合成対象言語 Pro5Lang

2.1 Pro5Lang の概要

再帰的強化学習アルゴリズム RGoal [10] [3][4] の行動価値関数を圧縮表現したものは、エージェントの行動を制御するプログラムと見なすことができる [11]。我々はこのプログラムを表現するプログラミング言語を Pro5Lang [2] と名づけ[2]、その言語仕様の設計・実装を進めている。行動価値関数の学習と圧縮を行うことは、Pro5Lang で書かれたプログラムをプログラム合成することに等しい。合成の性能を上げるためには、Pro5Lang の言語仕様を適切に設計する必要がある。

Pro5Lang は論理型言語（数理論理学を基礎にしたプログラミング言語）と機械語（コンピューターを構成する論理回路が直接解釈実行できる言語）の特徴を合わせ持ったプログラミング言語である。情報の記憶場所としてレジスタと連想記憶装置を有し、そこに証明済みの命題を記憶する。

Pro5Lang のプログラムは**行動ルール**の集合として定義される。Pro5Lang のプロトタイプ実装では、行動ルールは rule(s,g,a) という構文で定義される。エージェントは、現在のレジスタの状態 s およびサブゴールの値 g とマッチする s,g のパターンを持つ行動ルールを、その価値の高さに応じて確率的に１つ選択し、その行動 a を実行する。行動ルールの価値の学習が進めばエージェントは報酬を最大化するように合理的に振舞うようになる。

エージェントが取り得る行動のうち、call, set, recall 命令はそれぞれサブルーチン呼び出し、レジスタの値の更新、宣言的知識の想起を実行するプリミティブである。他に、環境の観測、物体や自分自身の移動、他者との対話などを行うプリミティブがある。

Pro5Lang の詳細については [2] を参照されたい。

2.2 Pro5Lang のメモリ構成

図１は現在のバージョンの Pro5Lang のメモリ構成である。汎用レジスタとして a レジスタと e レジスタがある。また、大容量メモリである連想記憶装置 [2] がある。連想記憶装置は、脳においてエピソード記憶を記憶する海馬に設計のヒントを得ている。

Pro5Lang は以下の点で機械語と類似している。

1. すべての命令（行動ルール）とデータ（証明済み命題）が統一された形式を持つ固定長の整数ベクトルで表現される。
2. 少数のレジスタと大容量のメモリを用いて情報処理を行う。
3. 固定した回路（論理回路または神経回路）で実現可能なほど言語仕様がシンプルである。

一方で、大容量メモリである連想記憶装置へのアクセスの仕方が機械語とは大きく異なる。Pro5Lang では、 a レジスタに値がセットされたびに、同じ値が連想記憶装置に自動的に追加される。連想記憶装置からの値の取り出しは検索クエリを引数とする recall 命令で行う。取り出された値は e レジスタにセットされる。

行動ルール rule(s,g,a) の定義時に指定する状態 s は、２つの汎用レジスタ（a と e）の値の組である。サブゴール g は、現在実行中のサブルーチンの終了条件を表現する値であり、call 命令によりセットされる。call 命令は再帰的に呼び出すことが可能で、そのためにサブゴールのスタックが存在する。状態 s が終了条件 g を満たすと、g の値は call 時にスタックに積まれた値に戻る。

2.3 Pro5Lang による推論

Pro5Lang のプログラムは、既知の知識を使って、環境の隠れた状態などについて推論することができる [2][12]。推論規則は行動ルール集合によって表現される。例えば推論規則 P,Q ⊢ R を用いて命題 R を証明するプログラムは、

rule(w(), w(a(R)), call(a(P)));
rule(w(a(P)), w(a(R)), call(a(Q)));
rule(w(a(Q)), w(a(R)), recall(e(P)));
rule(w(a(Q),e(P)), w(a(R)), set(a(R)));

となる。このプログラムはまず P, Q を証明するサブルーチンを呼び出し、それが完了すれば命題 R が証明されたものとみなして a レジスタに R をセットする。３行目の recall 命令は、２行目のサブルーチン呼び出しで破壊された「P が証明済みである」ことを意味するレジスタの値を復元するために必要である。このプログラムは、レジスタ名の指定などを省略した疑似コードで書けば

rule(_, R, call(P))
rule(P, R, call(Q))
rule(Q, R, recall(P))
rule({P,Q}, R, set(R))

となる。図 5、図 6、図 7 の疑似コードはこの記法を用いている。同様の推論規則をたくさん持つことで、Prolog 言語のような後ろ向き連鎖[3]による証明探索を実行し、様々な命題を推論することができる。

[2] Probabilistic Proven-Proposition Processing Programming Language （確率的証明済み命題処理プログラミング言語）の略。

[3] ゴールから後ろ向きに推論規則を適用していき証明を完成させる手法。

Pro5Lang プログラムを定理証明システムとみなす場合、レジスタの値と連想記憶装置の中の値はすべて**証明済み命題**である。プログラムの実行は、証明済み命題を用いて新たな命題を証明し、a レジスタにセットすると同時に連想記憶装置に追加する、という動作の繰り返しである。普通のプログラミング言語では任意の値を任意のメモリアドレスに自由に書き込めるのに対し、Pro5Lang のメモリアクセスには非常に強い制約があることになる。この強い制約は、エージェントによる自律的なプログラム合成の際に解の探索空間を狭くし、合成をしやすくすることにつながると考えている。(この制約と大規模言語モデルの類似性について 5.4 節で議論する。)

証明に用いる行動ルールの「正しさ」は、行動ルールの価値(ゴールに到達するまでのコスト)として、強化学習アルゴリズムを使って学習する [12]。価値の学習は、センサー情報と矛盾する命題を set する動作を検出すると推論をやり直すという機構のもとで行われる。この機構により、エージェントがいる環境において正しくない推論規則の価値は下がり、選択されなくなってゆく。

以上のように Pro5Lang は論理型言語に似た特徴を持っている。一方で、代表的な論理型言語である Prolog 言語とは、以下の点で違いがある。

1. Pro5Lang の動作の基盤は数理論理学よりもむしろ強化学習である。推論の目的はあくまで報酬最大化であり、正しいがコストのかかる推論よりも、厳密には正しくないがコストの低い推論の方が選択されやすい。また、学習の途中においては間違った推論も行われ得る。

2. Prolog 言語は言語処理系自体が定理証明システムだが、Pro5Lang は汎用の手続き型言語であり、エージェントが自身の経験から自律的に定理証明プログラムを獲得する。

3. 真偽値だけではなく確率分布や価値が扱われる。

4. 推論や知識表現において、矛盾、unknown を表す値 、例外を許容する。

5. 証明探索は非決定論的に行われる。

3 Pro5Lang における知識表現形式

3.1 知識要素

図 2 は本稿で提案する Pro5Lang における知識表現形式である。Pro5Lang が処理する情報の基本単位を**知識要素** (knowledge element) と呼ぶことにする。これは計算機におけるワードに相当する[4]。レジスタや連想記憶装置に入れられる値(証明済み命題)はすべて図 2 に示される知識要素の形式をした値である。

知識要素は、自然言語の複文程度の情報を持つ命題を表現できる。現在の実装では、１つの知識要素は１つの接続詞と３つの**節**から構成される。個々の節は、状況、動詞、主語、といった**スロット**を持つ。個々のスロットの値は、**シンボル** である。(現バージョンでは未実装だが、シンボルは実際には 3.4 節で述べる EID で表現することを想定している。)

以前述べた知識表現形式 [2] から拡張された点は、状況、極性、 EID 、感覚イメージの導入である。それぞれについて以下の節で説明する。

3.2 状況

状況は、文の真偽値を決めるために必要な情報である。一般に文の真偽は文そのものだけからは決まらない。例えば「火星人が地球を攻めてきた」という文は、現実世界では偽、小説「宇宙戦争」の中では真、ラジオドラマ「宇宙戦争」を本当のことだと信じた人の心の中では真である。

Pro5Lang では、文の内容とその文が成り立っている状況を必ずひとまとめにして扱う。これにより、情報処理の最小単位である知識要素が自己完結的に意味を持つことになる。このことは、プログラム合成、実行順序の柔軟な変更、記憶域管理を容易にする。詳しくは 5.1 節で議論する。

本稿では図 2 のように、状況は節ごとに指定し、命題全体には指定しない、という設計とした。この設計のもとで 4 節のテスト用タスクが記述できており、現在のところ問題ないと考えているが、他の表現形式の可能性を排除するものではない。他の可能性については 5.2 節で議論する。

本稿の疑似コードでは、「状況 s のもとで節 c が成り立つ」という命題を $s \models c$ と表記する。(s supports c と読む。) この記法は状況意味論 [7] で使われているものを引き継いでいる。

3.3 極性

極性 (polarity) は、節が表現する内容が自然言語でいう肯定文か否定文かを示す符号である。この用語も状況意味論 [7] で使われているものを引き継いでいる。現在の実装では極性スロットには true か false という２つの記号のいずれかが入れられ、それぞれ肯定文、否定文を表す。例えば「きのう学校にネコがいた」とい

[4] 例えば 64bit CPU においては１ワードは 64bit であり、データの処理を 64bit 単位で行う。

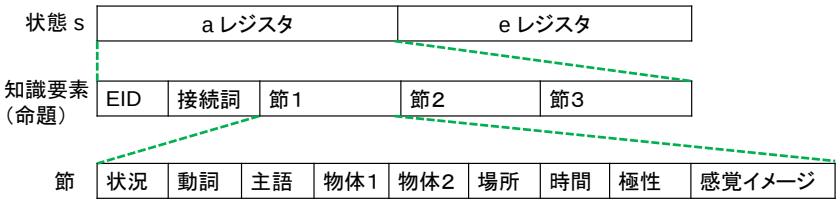

図 2: 知識要素の構造。知識要素は Pro5Lang の情報処理の基本単位である。固定長ベクトルを用いて、複文程度の情報を持つ命題を表現する。

う内容の節は

(Actual, Exists, Cat, ···. School, Yesterday, **true**)

「きのう学校にネコがいなかった」という内容の節は

(Actual, Exists, Cat, ···. School, Yesterday, **false**)

というベクトルで表現される。（“Actual” は「現実の状況」を表すシンボルである。）サブルーチン呼び出しにおいては極性に値 “+” を指定することができる。これは unknown 以外の値にマッチする特殊な値である。例えば「きのう学校にネコがいたかどうか」を推論するサブルーチン呼び出しは、

call(<···, Actual, Exists, Cat, ···. School, Yesterday, +, ···>)

となる。

3.4 EID を使ったポインタとシンボル

計算機のプログラミング言語におけるポインタとシンボルに相当する機能が、脳内でどう実現され得るかについての、１つの仮説をこの節で述べる。なお、ポインタの機構は現在の Pro5Lang では未実装である。

複雑な情報を表現するには、ポインタようなものが必須であろう。ヒトが扱う情報は一般に複雑な構造を持っている。文は入れ子になった埋め込み構造を持つことがあるし、「あの男」などの前方照応によって他の文の中身を参照することもある。一方で脳は固定した神経回路で構成されているため、一度に扱う情報は固定長のベクトルであると考えるのが自然である。そうすると、複雑な情報は固定長のデータ構造を、ポインタでつないだ形で表現せざるを得ないだろう。しかしながら、脳においては知識はまず海馬に記憶され、それから時間をかけて大脳皮質に移動して定着するため、計算機のようにメモリアドレスをポインタとして使えるのかどうかについては疑問がある。

提案する仮説では、ポインタは**知識要素ＩＤ（EID**, knowledge Element IDentifier）と呼ぶもので表現する。図 2 における EID のスロットの値が、知識要素 ID である。Pro5Lang では a レジスタに値を set するとその値は自動的に連想記憶装置に記憶されるが、同時に、a レジスタの内容を圧縮表現した特徴ベクトルを EID として付加するものとする（図 3)。EID を使って連想記憶装置から特定の知識要素を想起できるので、EID は実質的にポインタの役割を果たせる。知識要素を EID に圧縮する回路は、大脳皮質上のオートエンコーダーとして実現できる。なお、a レジスタ内の記号表現（シンボル列）自体も、感覚イメージをオートエンコーダーで圧縮することで獲得されるものと想定する。大脳皮質の回路は双方向であるから、推論結果などによって記号表現だけが得られている場合でも、そこから感覚イメージを想起して、知識要素を構築することが可能である。

3.1 節で知識要素の各スロットの値はシンボルであると述べたが、本節の仮説ではシンボルの実体も EID である。エージェントがある概念を思い浮かべたときに選択される知識要素を**代表知識要素**と呼ぶことにする。シンボルの実体は代表知識要素の EID である。例えば「犬」という抽象概念を表すシンボル Dog の実体は、犬についての代表的な感覚イメージを圧縮した特徴ベクトルで表現される。「白い犬が黒い猫を追いかけた」という知識要素は、節の主語スロットに「白い犬がいた」という知識要素の EID を、目的語となる物体１スロットに「黒い猫がいた」という知識要素の EID を入れることで表現可能になる（図 4)。

EID の中の一部の特徴をワイルドカード（記号 “_” で表す）にすることで抽象概念の間の包含関係も表現可能である。ワイルドカードを指定する特徴が多いほど、より抽象的な概念となる。例えば生物が (羽がある、肉を食べる、泳ぐ) という３つの特徴量で表現されている場合、「鳥」という概念は (1, _, _) 、「肉食動物」という概念は (_, 1, _) 、「生物」という最も抽象的な概念は (_, _, _) という特徴ベクトルで表現可能となる。Pro5Lang ではパターンマッチにより手続き的知識（行動ルール）および宣言的知識（知識要素）を検索するが、検索の際にワイルドカードを使うことで、継承関係に応じた知識の再利用が可能になる。例えば「生物はいつか死ぬ」という１つの宣言的知識が、「鳥はいつか死ぬ」「肉食動物はいつか死ぬ」という宣言的知識も自動的に兼ねることになる。Pro5Lang における知識の検索では「最も特殊なパターンをもつもの」が選択されるので、知識を継承せず上書きすることも可能である。この継承機構は 4 節の例で必要になるような、

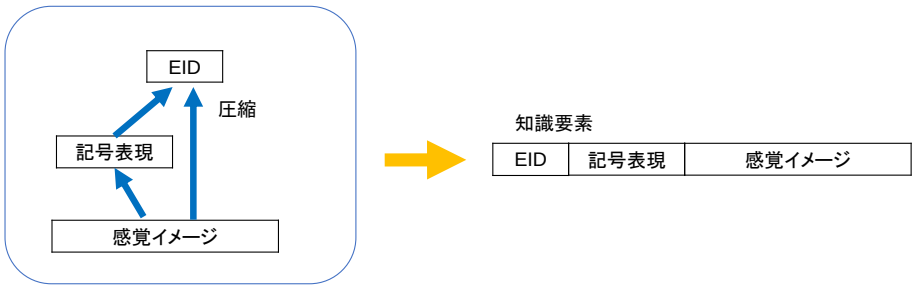

図 3: EID は、知識要素の内容を情報圧縮するオートエンコーダーにより瞬時に生成可能である。

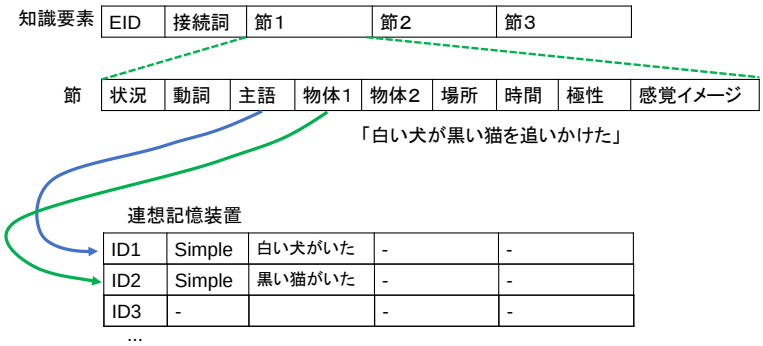

図 4: 連想記憶装置内の記憶には、ほぼユニークな EID が付加される。（ただしプロトタイプ版では未実装。）知識要素内の各スロットの値は、表したい概念を代表する知識要素の EID である。図中の “Simple” は単文を表す便宜的な接続詞。

状況の継承においても利用可能である。

以上をまとめると EID は以下の役割を持つと言える。

1. C 言語や Java 言語などにおけるポインタ。あるデータ構造の中から大容量メモリ内にある別のオブジェクトを参照できる。
2. ハッシュ表におけるハッシュ値。大容量の表から値を瞬時に取り出せる。
3. 自然言語処理における embedding 。ある程度複雑な情報を固定長ベクトルに埋め込める。
4. 記号 AI におけるシンボル。永続的に一意性が保証される値であり、同値かどうかの比較が瞬時に行える。
5. 特徴ベクトル。ワイルドカードを使うことによって概念間の包含関係と知識の継承が表現できる。

なお、この節で述べた機構だけでは EID の衝突など様々な問題が発生する。この問題は、エージェントが自律的に獲得した Pro5Lang プログラムが解決するものと考えている。詳しくは 5.3 節で説明する。

3.5 感覚イメージ

これもプロトタイプでは未実装であるが、節ごとに非記号的な感覚イメージの情報を含めるべきであると考えている。これは、脳でいえば感覚連合野の発火パターンに相当する。完全に記号化できないような、アナログ的な視覚、聴覚、触覚などのイメージが、時系列的な文脈情報も含めて、固定長の整数ベクトルで表現されたものがこのスロットの値になると想定している。

3.6 知識要素の脳内での実現方法

提案する知識要素の表現形式がヒトの脳内でどう実現されると想定しているかについて簡単に述べておく。

これまで述べたように、知識要素は接続詞、状況、主語、動詞などのスロットから構成される固定長のシンボルのベクトルである。個々のシンボルの実体は EID であり、EID の実体は固定長の特徴ベクトルである。

我々は、脳内の特徴ベクトルの要素は非負整数値であると考えている。特徴ベクトルの個々の要素は大脳皮質におけるマクロコラムに対応し、個々の要素がとり得る値は排他的に活性化するミニコラムに対応すると考えている [13]。１つのマクロコラム内にミニコラムは 100 個ほどあり、それが 1 〜 100 の整数値を表

現すると考える。値 unknown は、マクロコラム内のどのミニコラムも活性化していない状態で表現する。

仮に a レジスタが大脳皮質の１つの領野で表現されると仮定しよう。（例えば２２野あたりが文の意味を表現していることを示唆する研究がある [14]。）１つの領野のマクロコラム数は約１万個である。レジスタ内の節の数を 3 個、スロット数を 10 個とすると１つのスロットは約 300 個のマクロコラム、すなわち約 300 次元の特徴ベクトルで実現されていることになり、十分な表現力があると思われる。

4 テスト用タスク

4.1 テスト用タスクの実装の目的

命題に「状況」を導入した結果、仮想的状況における知識と現実の状況における知識の両方を用いた推論が可能になる。そのことを確認するために、Pro5Lang のプロトタイプ実装の上で、簡単なテスト用タスクを解くプログラムを手で書いて動作確認した。

4.2 背理法による証明

以下のような、ヒトの子どもでも容易に行える推論のシナリオを考える。

- 弟が「おかしを食べてない」と言っているが、実際には食べたことを証明したい。
- ゴミ箱におかしのゴミがあるのを見たとする。
- 「おかしを食べてないならばゴミはない」という知識があるものとする。
- 「もしおかしを食べてないならゴミがないはずだが、ゴミを見たので矛盾する。だから実際は食べたはずだ。」と推論する。

ヒトがこのような証明を行うとき、脳内ではどのような処理が行われるのだろうか？図 5、図 6 は Pro5Lang のプログラムを用いて、脳内の処理の１つの可能性を表現したものである。以下にプログラムの内容について簡単に説明する。

図 5 では背理法のための汎用的な２つのサブルーチンを定義している。この中で使われている PbyC(s $\models$ g), Assumed(s, ss, c), Inherits(s, ss) はプログラムを見やすくするためのマクロであり、いずれも知識要素の表現にマクロ展開される。例えば PbyC(c) は「節 c が背理法で証明された」という命題であり、実際には PbyC という特殊な接続詞と節 c から構成される知識要素にマクロ展開される。not(g) もまたマクロであり、これは節 g の中の極性を反転させたものにマクロ展開される。（g は節のリテラル表現で、極性には変数ではなく定数 true か false が書かれているものとする。）

図 6 では命題 A を証明するために必要となる問題領域に固有のサブルーチン２つと宣言的知識２つを定義している。k(...) はエージェントがプログラムの実行開始前からあらかじめ持っている宣言的知識を定義する構文である。最後の taskGoal(...) という行は、このテスト用タスクのゴールを定義している。

このテスト用タスクを実行すると、まずサブルーチン PbyC(Actual $\models$ A) の呼び出しが実行される。呼び出されたサブルーチン側は、与えられた状況を継承する**サブ状況** ss を生成し、ss のもとで命題 not(A) を仮定したあと、ss において矛盾が生じることを証明し、それが成功すれば背理法による証明が成功したものとしてサブルーチンを終了する。

この証明の過程で、命題 ss $\models$ Contradiction を証明するサブルーチン（図 6）が呼び出される。このサブルーチンは、問題領域固有の宣言的知識と手続き的知識（行動ルール）を用いて実際に矛盾が生じることの証明を試み、証明が成功すればサブルーチンを終了する。（矛盾を証明するには様々な戦略、様々なシナリオが考えられるが、図 6 のプログラムでは１つのシナリオのみが実装されている。）

図 6 の２つ目のサブルーチンは、状況の継承関係を用いて命題 ss $\models$ G を証明するためのものである。もし状況 ss が状況 s のサブ状況であり、かつ、もし s $\models$ G が成り立つなら ss $\models$ G も成り立つという推論を行っている。エージェントは Actual $\models$ G という宣言的知識は持っていても ss $\models$ G という知識は最初は持っていないため、このような処理が必要となる。（なお、3.4 節で述べたように、ヒトの脳内では特徴ベクトルに対してワイルドカードを使うことで継承関係を直接処理することも可能であろう。）

4.3 場合分けによる証明

この節では以下の推論のシナリオを考える。

- あしたは日曜日だが学校に行く必要があることを証明したい。
- 以下の知識があるものとする。
  - あした雨なら映画観賞会がある。
  - 映画観賞会があるなら学校に行く必要がある。
  - あした雨でないなら運動会がある。
  - 運動会があるなら学校に行く必要がある。

図 7 は、このときにヒトの脳内で行われるかもしれない情報処理を、Pro5Lang を用いて表現したものである。以下にプログラムの内容について簡単に説明する。

Case(s,ss,c,g) は「状況 s を継承するサブ状況 ss において節 c を仮定すると節 g が成り立つ」という命題である。実際には Case(s,ss,c,g) マクロであり、特別な接続詞 Case と 3 つの節 Inherits(s, ss), c, g から構成される知識要素にマクロ展開される。

最初にサブルーチン Actual $\models$ S が呼び出されると、雨が降る場合と雨が降らない場合とで場合分けし、それぞれの場合に S が成り立つことを証明する。どちらの証明も成功したならば、 Actual $\models$ S が証明され、タスクのゴールが達成される。

4.4 場合分けにおいて例外的状況がある場合

前節のプログラムに知識を追加し、洪水で学校がなくなった状況 Flood において S が成り立つかどうか考えたいとする。それには、下記の「雨が降れば映画観賞会が開かれる」というデフォルトの推論規則：

rule(s $\models$ R, s $\models$ S, s $\models$ E1)

に加え、以下のような「雨が降っていても洪水が起きた状況であれば映画観賞会は開かれない」という例外的な推論規則：

rule(Flood $\models$ R, Flood $\models$ S, Flood $\models$ not(E1))

を追加すればよいように思われる。Pro5Lang の行動選択においては「最も特殊」な行動ルールが選択されることになっているため、例外ルールが優先的に選択され、call(Flood $\models$ S) という証明は成功しなくなるはずである。

ところが現在の Pro5Lang 実装でこれを動かしたところ、証明が成功してしまう。上記デフォルトルールが呼ばれるとき Flood ではなく Flood のサブ状況なので例外ルールにマッチしないためである。しかしこの振る舞いは脳のモデルとしては間違っておらず、そのような「見落とし」はヒトでも起こり得ることである。プログラム合成がもし理想的に収束すれば、他の必要な行動ルールが新たに獲得され、このような問題は起きなくなると予想される。また 3.4 節で述べた特徴ベクトルとワイルドカードを使った直接的な継承処理であれば、問題が起きない可能性がある。

5 議論

5.1 知識要素が自己完結的である利点

文の内容とその文が成り立っている状況をひとまとめにすることで、情報処理の最小単位である知識要素は、自己完結的に意味を持つことになる。このことは、プログラム合成、実行順序の柔軟な変更、記憶域管理を容易にする。それぞれについて以下に説明する。

まず、知識要素が自己完結的であるならば、rule(s,g,a) という形をした行動ルールもまた、それ単独で意味を持つことになり、強化学習を用いた価値の学習が容易になる。つまり、強化学習を用いたプログラム合成が容易になる。（一般のプログラミング言語においては、ある程度のまとまりを持ったコードでないと、タスクに関係した意味を持たず価値の評価が難しい。）

同様に、行動ルールの実行の各ステップもそれ自体で意味を持つ。特に推論タスクにおいては、実行の各ステップは、証明済み命題から他の命題を推論することなので、実行順序の自由度が高い。このことは、エージェントにとって時間に余裕があるときに将来の行動計画を立てたり、割り込みで中断された推論タスクをあとで再開したりと言った、実行順序の柔軟な変更が容易に行えることを意味する。

知識要素が自己完結的であることは、記憶域管理も容易にする。連想記憶装置の記憶容量は有限であるから、適宜忘却が必要となる。知識要素が自己完結的であれば、その中からランダムに選んで忘却してもシステムの状態の一貫性が破綻しにくいことになる。さらに、 Pro5Lang では連想記憶装置の中身は証明済み命題に限定されているため、必要な知識要素が忘却された場合も、他の知識から証明しなおせる場合も多い。

5.2 知識要素の他の形式の可能性

本稿では図 2 のように、状況は節ごとに指定し、命題全体には指定しない、という設計としたが、他の表現形式の可能性を排除するものではない。（そもそもヒトでは個体ごとに内部の表現形式は多少異なっている可能性もある。）

現在の表現形式では、命題全体の状況の指定がないことから「状況 s において『もし c1 ならば c2 である』」という命題が直接的には表現できないことになる。しかしポインタを用いて複数の知識要素を組み合わせればこのような複雑な命題も表現可能であろう。

現在の実装では節の数を 3 つとしたが、フルセットの節を 3 つ持つ必要はなく、もう少し簡略化された表現でも十分かもしれない。

さらに、知識要素の中の節を 1 つだけとする表現形式も考えられる。ポインタがあるならば、 Lisp 言語のように、スロットが 2 つあればあらゆるデータ構造が原理的には表現可能である。しかし知識要素がシンプルすぎると、それを処理するプログラムの方が複雑になり、プログラム合成が難しくなるだろう。

```
// 背理法 (proof by contradiction) で g を証明するサブルーチン
pbcg = PbyC(s ⊨ g)
rule(_, pbcg, call(Assumed(s, +, not(g))))
rule(Assumed(s, ss, not(g)), pbcg, set(ss ⊨ not(g)))
rule(ss ⊨ not(g), pbcg, call(ss ⊨ Contradiction)) // 矛盾が起きることを証明
rule(ss ⊨ Contradiction, pbcg, recall(Assumed(s, ss, not(g))))
rule({ss ⊨ Contradiction, Assumed(s, ss, not(g))}, pbcg, set(PbyC(s ⊨ g)))

// s を継承する状況 ss で節 c を仮定するサブルーチン
ag = Assumed(s, +, c)
rule(_, ag, primitive(NewSituation,s))
rule(Inherits(s, ss), ag, set(ss ⊨ c)) // 新規に作った状況 ss で c を仮定
rule(ss ⊨ c, ag, recall(Inherits(s, ss)))
rule({ss ⊨ c, Inherits(s, ss)}, ag, set(Assumed(s, ss, c)))
```

図 5: 背理法で証明する Pro5Lang プログラムの疑似コード。小文字で始まる記号は変数、大文字で始まる記号は定数。“_” は任意の値に、 “+” は unknown 以外の値にマッチするワイルドカード。

```
// 任意の状況 ss で ss ⊨ not(A) ならば ss ⊨ not(G) が成り立つという宣言的知識
k(<If, ss ⊨ not(A), ss ⊨ not(G)>)

// ss のもとで矛盾が生じることを証明（PbyC から呼ばれる）
cg = ss ⊨ Contradiction
rule(ss ⊨ not(A), cg, recall(<If, ss ⊨ not(A), ss ⊨ +>))
rule({ss ⊨ not(A), <If, ss ⊨ not(A), ss ⊨ not(G)> }, cg, set(ss ⊨ not(G)))
rule(ss ⊨ not(G), cg, call(ss ⊨ G))
rule(ss ⊨ G, cg, recall(ss ⊨ not(G)))
rule({ss ⊨ G, ss ⊨ not(G)}, cg, set(ss ⊨ Contradiction))

// 「Actual （現実の状況）において G が成り立つ」を意味する宣言的知識
k(Actual ⊨ G)

// 状況 ss において G が成り立つことを証明するサブルーチン
ssgg = ss ⊨ G
rule(_, ssgg, recall(Inherits(+, ss)))
rule(Inherits(s, ss), ssgg, recall(s ⊨ G))
rule({Inherits(s, ss), s ⊨ G}, ssgg, set(ss ⊨ G))

// タスクのゴール：背理法により A が成り立つことを証明
taskGoal(PbyC(Actual ⊨ A))
```

図 6: 背理法で命題 A を証明する Pro5Lang プログラムの疑似コード。Actual: 現実の状況、A: 「おかしを食べた」、 G: 「おかしのゴミがある」。

```
// 状況 s を継承する状況 ss において、
// c を仮定すると g が成り立つことを証明するサブルーチン
cgg = Case(s, +, c, g)
rule(_, cgg, call(Assumed(s, +, c)))
rule(Assumed(s,ss,c), cgg, set(ss ⊨ c))
rule(ss ⊨ c, cgg, call(ss ⊨ g))
rule(ss ⊨ g, cgg, recall(Assumed(s,ss,c)))
rule({ss ⊨ g, Assumed(s,ss,c)}, cgg, set(Case(s, ss, c, g)))

// 状況 s で S が成り立つことを、
// R の時と not(R) の時で場合分けして証明するサブルーチン
cg = s ⊨ S
rule(_, cg, call(Case(s, +, R,S)))
rule(Case(s, _, R,S), cg, call(Case(s, +, not(R),S)))
rule(Case(s, _, not(R),S), cg, recall(Case(s, +, R,S)))
rule({Case(s, _, not(R),S), Case(s, _, R,S)}, cg, set(s ⊨ S))

// S が成り立つことを証明するサブルーチン
sg = s ⊨ S
rule(s ⊨ R, sg, set(s ⊨ E1))
rule(s ⊨ E1, sg, set(s ⊨ S))
rule(s ⊨ not(R), sg, set(s ⊨ E2))
rule(s ⊨ E2, sg, set(s ⊨ S))

// タスクのゴール： S が成り立つことを証明
taskGoal(Actual ⊨ S)
```

図 7: 場合分けで命題 S を証明する Pro5Lang プログラムの疑似コード。S:「明日学校に行く必要がある」、 R:「明日は雨」、E1:「学校でイベント E1 がある」、E2:「学校でイベント E2 がある」。

5.3 EID に関して起こり得る問題

EID を生成する図 3 の機構だけでは、以下の問題が起こり得る。

- EID の衝突。見た目で区別付かない 2 つの物体や人物の区別、よく似た 2 つのエピソードの区別など。
- EID の変化。知識が増えることによって対象の概念の特徴ベクトルが変化する場合など。
- EID の統合。異なる概念だと思っていたものが実は同一のものであると知ったときなど。
- EID の分離。同じものだと信じていたものが別のものだとわかったときなど。

これらすべての問題は、おそらく経験で獲得したプログラムによって対処可能であろう。RGoal の性質により、のちのち役立つように脳内の知識を整理するようなプログラムが、自律的に獲得されるはずである。

例として EID の衝突の問題について考える。例えば目の前にたくさんの同じ形をしたコップがあるとき、個々のコップの特徴ベクトルは完全に同一のものになってしまうだろう。また、似た状況で会った似た顔の人も、同一の特徴ベクトルになる可能性がある。この問題を解決するために、おそらく脳は、 EID の一意性を、経験で獲得したプログラムを用いて保証するのだろう。十分に経験を積んだヒトの個体であれば、同じ特徴の物体が複数ある時にはのちのち情報処理が混乱することを予見し、対策を講じるはずである。具体的には、「自分の右側にあるコップ」とか「水が半分になったコップ」のように、あとで判別可能な特徴を能動的に付与し、異なる感覚イメージを持たせることで、圧縮した特徴ベクトルも異なるものにできるはずである。また、見た目が似た人物であっても、名前を含めて EID を作れば区別可能になる。

EID の一意性を保証するプログラムは、エージェントが報酬最大化原理のもとで自律的に獲得するだろう。一意性の保証がうまくいく方が、うまくいかなくて似た物体を間違って扱ってしまうよりも、より高い報酬を獲得できる、もしくはより低いコストでタスクを終了できると考えられるからである。

ID の一意性の保証は計算機上でも簡単な問題ではない。Java 言語ではポインタをオブジェクト ID の代わりに用いることができるが、背後でガーベージコレクターという複雑な記憶域管理プログラムが正しく動作することで、はじめて ID の一意性が保証される。また RDF (Resource Description Framework) という知識表現形式においては、ノードの ID は URI を用いて表現され、その一意性は知識の設計者が注意深く保証する必要がある。これらのことを踏まえると、脳においても ID の一意性が何か未知の巧妙な機構で自動的に保証されると考えるよりも、経験で獲得されたプログラムで保証されると考える方が自然に思われる。

起こり得る問題のもう 1 つの例として、物体の EID の変化について考える。例えば初対面の人を表現する特徴ベクトルは、その後の付き合いなどで情報が増えるにつれ、大きく変化していくはずである。おそらく脳は、 EID が変化した時、経験で獲得したプログラムを用いて能動的に知識を整理するのだろう。十分に経験を積んだヒトの個体であれば、物体の特徴ベクトルの変化に気づいたとき、のちのち情報処理が混乱することを予見し、知識の整理を開始するはずである。それには、宣言的知識の中にある古い EID をすべて新しいものに付け替える必要がある。それを行うサブルーチンは、例えば以下のようなものになる。

g = Replaced(old,new)
rule(_, g, recall(<..., old, ...>)))
rule(<..., old, ...>, g, set(<..., new, ...>))

このサブルーチンは、古い EID を使った記憶をランダムに想起し、新しい EID に置き換えて記憶しなおす。連想記憶の想起で新しい記憶が優先されると仮定すれば、付け替える前の古い EID を使った記憶は想起されなくなる。エージェントは時間的余裕のあるときにこのサブルーチンの実行を行えばよい。経験を積めば、エージェントはこのような単純なプログラムよりもより効率的な「EID 付け替えプログラム」を獲得可能だろう。

一般にヒトは、このような知識の整理を前もって行うことで、将来のタスクにおける報酬最大化を図っているのではないだろうか。将来の未知の状況に備えて前もって知識の整理をするという行動は長期的な報酬を最大化する合理的行動であり、RGoal の学習アルゴリズム [4] により学習したサブルーチンがうまく汎化すれば、自然に創発するはずである。

5.4 連想記憶装置と大規模言語モデル

2.2 節、2.3 節で述べたように、Pro5Lang では a レジスタにセットした証明済み命題が連想記憶装置にも自動的に追加される。エージェントは報酬最大化を目的として、証明済みの命題から別の命題を推論し、連想記憶装置に追加するという動作を繰り返す。連想記憶装置の中身はエージェントがそれまでに推論した命題の集合であり、Pro5Lang プログラムの推論の履歴とも言える。

Pro5Lang プログラムのこの振る舞いは、ChatGPT[15] のような大規模言語モデル (LLM) の振る舞いに類似している。LLM はそれまでの対話履歴から次に出力す

べきトークンを推論し、そのトークンを対話履歴に追加する、ということを繰り返す。ChatGPT は Reinforcement Learning from Human Feedback (RLHF) でファインチューニングされているため、推論の目的は報酬最大化である。LLM を汎用計算機とみなすならば、対話履歴はその計算機の内部状態であると同時に実行履歴である。対話履歴からの情報の検索はおそらく Transformer の self-attention を用いて行われるが、これはメモリのアドレスではなく中身に基づいたアクセスであるという点で、連想記憶の想起と類似している。この汎用計算機を動かす「プログラム」は、大量のテキストデータを学習することで獲得され、LLM のパラメタの値という形で保持されている。

つまり、Pro5Lang プログラムも LLM も、報酬最大化を目的として合成された推論プログラムであり、そのプログラムはそれまでの推論履歴を参照してあらたな推論を行い、結果を推論履歴に追加するという動作を繰り返す。

筆者は主に神経科学的知見との整合性を重視して Pro5Lang とそのメモリ機構を設計したが、それが今日人間に近い知能を発揮しつつある LLM の機構と類似点を持つことは興味深い。LLM の成功は、プログラム合成システムとしての Pro5Lang の設計の有望さを示唆しているかもしれない。

一方で、 Pro5Lang を設計している筆者から見て現状の LLM には問題点があるように見える。LLM は推論結果をテキストの形をした対話履歴でしか保持しないため、知識の追加と検索が非効率的かつ不正確である。このことは、学習に膨大なテキストデータを用いたにも関わらず、複雑な行動計画、計算、推論の能力は十分でない [16] ことにつながっているのではないだろうか。Pro5Lang のように、行動計画や推論に直接的に特化した知識表現を支援することが、汎用人工知能の実現には必要なのではないだろうか。

6 関連研究

6.1 状況理論・状況意味論

状況理論・状況意味論 [6][7] は、理論言語学や自然言語処理の分野におけるかつての試みであり、状況という概念を導入して文の意味を形式的に表現し、現実世界における推論の基盤にしようとした。

Pro5Lang は状況意味論からいくつか記法や用語を借用したが、状況意味論と Pro5Lang との詳細な関係は現時点では明らかでない。以下に現時点での考えを述べる。

状況意味論では、自然言語の文の意味を数学的実体に厳密に対応付けようとしているように見える。Pro5Lang ではデータの意味は解釈プログラムが解釈するのであって、それは「解釈プログラムの動作そのもの」以上に抽象化しようのない、本質的に複雑なものである。他者が何か発話したら、その発話したという事実をもとに、報酬最大化を目的としてさまざまな命題を推論する（すなわち解釈する）、というのが Pro5Lang プログラムの動作であり、文を決められたルールで１つの数学的実体に変換するようなことはしない。また、処理プログラムは個体ごとに異なるため、同じ文であっても個体ごとに異なる意味で解釈されるのが普通であり、個体とは独立して存在する「意味」は想定していない。以上のように、 Pro5Lang は「自然言語の意味を数学的実体で表現する」という思想とは根本的に異なる思想で設計されている。

6.2 数理論理学における自然演繹

数理論理学とは数学の一分野であり、人間が行う推論や証明の過程を定式化して、研究の対象とする学問である。

人間が背理法や場合分けなどを用いた証明をする際には、恣意的で一時的な仮定の導入が行われる。自然演繹は、そのような証明を形式化する目的で、ヤシュコウスキ [8] とゲンツェン [9] が 1934 年に独立に開発した。

ゲンツェンはのちに、自然演繹を研究する道具としてシークエント計算という手法を生み出した。シークエント計算では前提と結論をひとまとめの式で表現して、それに対して一段メタな推論規則を適用する。十分な表現力を持ちつつも、仮定の追加・削除のための特別な仕掛けが不要な、シンプルな枠組みになっている。

Pro5Lang の知識要素は状況と文をひとまとめに扱っており、その点は前提と結論をひとまとめに扱うシークエント計算と類似している。このことは、 Pro5Lang の知識表現形式と推論機構が、ヒトの推論を再現するための必要十分な能力を持つことの傍証と言えるだろう。

7 まとめと今後

本稿では、プログラム合成対象言語 Pro5Lang の知識表現形式を提案した。これはヒトの脳内における知識表現方法の１つの仮説でもある。情報処理の基本単位は知識要素という固定長ベクトルであり、状況を表すスロットを持つ。この知識表現形式を用いて背理法と場合分けを用いた推論を行うテストプログラムを記述し、言語の表現力について確認した。また、シンボルやポインタのような機能を実現するために、EID （知識要素 ID）を導入する方法も提案し、起こり得る問題と対処方法について考察した。

Pro5Lang を中核技術とする脳型 AGI アーキテクチャを効率的に動作させるためには、解決すべき小さな問題はまだ多くあるが、アーキテクチャの全体像については、これでほぼ固まったと考えている。

謝辞

本研究は JSPS 科研費 JP22K12188 の助成を受けたものです。

参考文献

[1] 一杉裕志. 報酬最大化原理にもとづく脳型ＡＧＩアーキテクチャの構想. 第 18 回 人工知能学会 汎用人工知能研究会 (SIG-AGI), 2021.

[2] 一杉裕志, 中田秀基, 高橋直人, 竹内泉, 佐野崇. 汎用人工知能のためのプログラム合成対象言語 Pro5Lang のエピソード記憶機構. 第 20 回 人工知能学会 汎用人工知能研究会 (SIG-AGI), 2022.

[3] 一杉裕志, 中田秀基, 高橋直人, 竹内泉, 佐野崇. 再帰的な階層型強化学習 RGoal へのサブルーチン例外終了機能の導入. 第 25 回 人工知能学会 汎用人工知能研究会 (SIG-AGI), 2023.

[4] 一杉裕志, 中田秀基, 高橋直人, 竹内泉, 佐野崇. モンテカルロ版 RGoal アルゴリズムの改良. 第 26 回 人工知能学会 汎用人工知能研究会 (SIG-AGI), 2024.

[5] 一杉裕志, 中田秀基, 高橋直人, 佐野崇. 物体操作に適したワーキングメモリを持つ汎用人工知能アーキテクチャの検討. 第 16 回 人工知能学会 汎用人工知能研究会 (SIG-AGI), 2020.

[6] Jon Barwise and John Perry. *Situations and Attitudes*. MIT Press, Cambridge, Mass., 1983.

[7] Keith Devlin. Situation theory and situation semantics. In Dov M. Gabbay and John Woods, editors, *Logic and the Modalities in the Twentieth Century*, Vol. 7 of *Handbook of the History of Logic*, pp. 601–664. North-Holland, 2006.

[8] Stanisław Jaśkowski. On the rules of suppositions in formal logic. In Stanisław Jaśkowski, editor, *?Itepmccall1967*. Oxford at the Clarendon Press, 1934.

[9] Gerhard Gentzen. Untersuchungen über das logische schließen. i. *Mathematische zeitschrift*, Vol. 35, , 1935.

[10] Yuuji Ichisugi, Naoto Takahashi, Hidemoto Nakada, and Takashi Sano. Hierarchical reinforcement learning with unlimited recursive subroutine calls. In *Artificial Neural Networks and Machine Learning – ICANN 2019: Deep Learning*, pp. 103–114, Cham, 2019.

[11] 一杉裕志, 高橋直人, 中田秀基, 佐野崇. 単一化の機構を利用した階層型強化学習のテーブル圧縮手法の検討. 第 10 回 人工知能学会 汎用人工知能研究会 (SIG-AGI), 2018.

[12] 一杉裕志, 中田秀基, 高橋直人, 佐野崇. 推論規則の価値を階層型強化学習 RGoal を用いて学習する手法の提案. 第 14 回 人工知能学会 汎用人工知能研究会 (SIG-AGI), 2020.

[13] 一杉裕志, 中田秀基, 高橋直人, 佐野崇. 脳の自律的プログラム合成機構のモデルに向けて： ２層ベイジアンネットによる記号処理命令の獲得・実行機構. 第 15 回 人工知能学会 汎用人工知能研究会 (SIG-AGI), 2020.

[14] Steven M Frankland and Joshua D Greene. An architecture for encoding sentence meaning in left mid-superior temporal cortex. *Proceedings of the National Academy of Sciences*, Vol. 112, No. 37, pp. 11732–11737, 2015.

[15] Introducing ChatGPT. `https://openai.com/blog/chatgpt/`, 2022.

[16] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*, 2023.